Postern: a Lean-verified access gateway for agentic data lakehouses

Secure Program Synthesis Hackathon 2026 — Track 3 research artifact

true

Abstract

We address access control for data lakehouses queried by LLM agents. Row- and column-level security in ACID-class databases does not transfer to this setting — per-source policies do not compose across heterogeneous ETL paths — and the deployed alternative, physical tenant segregation, recovers safety only by forfeiting the cross-source joins that motivate the lakehouse. When upstream ACLs collapse to a single ingest IAM role at materialisation, an indirect-prompt-injected agent inherits the *union* of the upstream principals' permissions rather than the *intersection* under the querying identity. We propose plan-level rewriting against a **Biscuit-Datalog** policy (Clever Cloud and contributors 2024) and present **Postern**, an artifact in three parts. The first is a Plan IR and a column-grant surface syntax that compiles to ground `right(principal, relation, column)` facts in the Horn fragment, with a rewriter rewrite : $\mathit{Catalog} \to \mathit{Policy} \to \mathit{Principal} \to \mathit{Plan} \to \mathit{Option\ Plan}$ mechanised in Lean~4 (Moura and Ullrich 2021). Nine rewriter theorems are proved without `sorry` — output-column and filter-predicate soundness, idempotence, policy monotonicity, schema subset, two no-new-column lemmas, and two refusal lemmas — with axioms bounded by `propext` and `Quot.sound`. The underlying Horn-fragment Datalog evaluator is partly mechanised: `eval_monotone` holds modulo one isolated obligation on `List.eraseDups.length`, while `eval_sound` and `eval_terminates` carry `sorryAx` obligations named in `CheckAxioms.lean`. The second component is a Rust capability-tracking layer inspired by Odersky et al. (Odersky et al. 2026): invariant brand lifetimes, sealed types, and opaque-receipt sinks. The third is a reference-conformance harness asserting byte-equivalence between the Rust rewriter and the Lean reference on 18 cases; migrating the gateway to consume `biscuit_auth::datalog::World` directly is in progress. We evaluate on the Kaggle `transactions-fraud-datasets` schema and identify Biscuit attenuation, audience, expiry, and key rotation; cross-relation joins; and differentially-private aggregation as the principal open problems.

Introduction

Production agentic systems increasingly read context from data-lakehouse stores at inference time, compacting heterogeneous sources into engine-agnostic columnar formats — Apache Arrow Parquet on S3-class object storage — queried by in-process engines such as DuckDB (Raasveldt and Mühleisen 2019), often co-located with on-prem or edge runtimes hosting local models. The access-control consequence is sharp: the entity issuing queries is no longer a human analyst on a stable role but an LLM agent mediated by tool-call protocols such as the Model Context Protocol (Anthropic 2024). Agent access is short-term, contextual, and task-oriented; role-based access control — and its temporally-scoped variant TRBAC — does not fit. Two existing responses each concede something. Per-engine row- and column-level security (PostgreSQL Global Development Group 2024) does not survive ETL into an engine-agnostic Parquet store, forcing per-source policy duplication that scales poorly with source count and churn. Physical tenant segregation across object-storage prefixes queried by disjoint engines restores safety but forfeits the cross-source joins that motivate the lakehouse in the first place. Neither composes: when channel ACLs, field-level security, and customer-scoped tokens collapse to a single ingest IAM role at materialisation, an indirect-prompt-injected agent inherits the *union* of the upstream principals' permissions rather than the *intersection* under its querying identity. We propose plan-level rewriting against a column-grant policy as a third point in this design space. **Postern**, the artifact we present, has three components: a Lean~4-mechanised plan rewriter, a Rust capability-tracking layer constraining the agent's downstream computation, and a reference-conformance harness binding the two.

Contributions

1. A Plan IR (*Scan*/*Project*/*Filter*), a column-grant policy language, and a rewriter, mechanised in Lean~4 (Moura and Ullrich 2021). The development comprises nine `sorry`-free theorems: output-column soundness, filter-predicate soundness, schema subset, idempotence under repeated application, monotonicity in the policy, two no-new-column lemmas, and explicit-refusal lemmas for unknown relations and for filter predicates over forbidden columns (§4). Axiom dependencies are audited per theorem and bounded by `propext` and `Quot.sound`; two theorems depend on none.
2. A Rust capability-tracking layer (`postern-guardrail`) implementing three composable mechanisms: sealed `Cap<'sc, C>` tokens whose construction is private to the crate, an invariant brand lifetime `'sc` enforced by `PhantomData<fn(&'sc ()) -> &'sc ()>` and gated by a universally-quantified scope combinator, and opaque-receipt sinks that consume both the `Cap` and the carrier `Tagged` without exposing the underlying value. Three lexical bypass attempts (forging `Cap`, projecting `Tagged::value`, escaping the brand) are pinned as `compile_fail`

doctests. The agent-facing surface is `no_std`-compatible (§3).

3. A Rust implementation of the rewriter (`postern-core`) structurally mirroring the Lean reference, and a reference-conformance harness (`postern-diff`) that asserts byte-equivalence between the Rust output and the Lean reference on a corpus of 18 hand-curated cases (15 accept, 3 refusal). We label the procedure *reference-conformance testing* rather than QuickCheck-style differential testing, reserving the latter term for property-based generation; the latter is among the open problems of §6.
4. A case study over the Kaggle `transactions-fraud-datasets` schema, with three principals (CRM, Card Operations, Fraud Risk) exercising PII redaction, cross-departmental refusal, and minimum-necessary disclosure (§5). Each row of the evaluation table corresponds to a corpus case driving the conformance harness.

Plan IR

We state the IR before the threat model so §2 may reference its operators directly. Plans are single-relation expressions built from three constructors:

$$\mathit{Plan} \ ::= \ \mathit{Scan}(r) \mid \mathit{Project}(\mathit{Plan}, \mathit{cs}) \mid \mathit{Filter}(\mathit{Plan}, c)$$

where $r \in \mathit{Relation}$, $c \in \mathit{Column}$, and $cs \in \mathit{List\ Column}$. We write $\sigma(q)$ for the output schema of plan q under a catalog *cat*, defined inductively: $\sigma(\mathit{Scan}(r)) = \mathit{cat}(r)$; $\sigma(\mathit{Project}(p, cs)) = \sigma(p) \cap cs$; $\sigma(\mathit{Filter}(p, c)) = \sigma(p)$. The asymmetry between *Project* (which alters the output schema) and *Filter* (which does not, despite reading c) is the source of the *filter side-channel* discussed in §2.

Threat model

The trusted computing base (TCB) consists of the gateway process itself, the catalog *cat* that it consults, the plan-to-executor lowering step that hands the rewritten plan to DuckDB, the principal extraction step that maps a verified capability token to a *Principal* string, and the DuckDB + Parquet store. All other parties are untrusted.

Component	Trust	Justification
LLM / agent planner	×	Susceptible to direct and indirect prompt injection.
Agent-generated code	×	Composes upstream-tainted context with downstream effects.
Capability tokens (Clever Cloud and contributors 2024)	~	Trust contingent on the gateway's signature verification.

Component	Trust	Justification
Gateway process	✓	Hosts the Lean-verified rewriter and the policy.
Catalog ($r \mapsto$ columns)	✓	Assumed bound to the physical Parquet schema (§6).
Plan-to-executor lowering	✓	The rewritten plan is assumed honoured literally by DuckDB.
Principal-string extraction	✓	A bug in token verification invalidates every theorem of §4.
DuckDB + Parquet store	✓	Standard storage-engine assumptions apply.

The attacks within scope of the formal model are: (i) over- projection of forbidden columns; (ii) filter on a forbidden column (the side-channel addressed by `rewrite_filter_sound`); (iii) scan of a relation absent from the catalog (addressed by `rewrite_refuses_unknown`); (iv) cross-departmental reach by a principal lacking matching grants; and (v) unknown principals, which we treat fail-closed via the empty-allow convention.

The following are deliberately out of scope and discussed in §6: aggregation and inference attacks; covert channels through latency or row-count observation; multi-relation joins; biscuit attenuation modelled inside the Lean proof; policy synthesis from natural language; and the planner-to-executor lowering step.

Design

Postern compiles a single policy artifact to plan-level enforcement.

```
flowchart LR
  agent[LLM agent] -->|MCP plan + biscuit| gw{{Postern gateway}}
  subgraph TCB
    gw -->|verify| bc[biscuit verifier]
    bc -->|principal| rw[Lean-extracted rewriter]
    pol[(policy)] --> rw
    cat[(catalog)] --> rw
    rw -->|Option Plan| exe[DuckDB / Polars]
  end
  exe --> agent
```

Policy

A policy is a list of **column-grants** $\langle p, r, C\rangle$: "principal p may read columns C on relation r". Multiple grants for the same (p, r) flat-union. Anything outside the union is denied — fail-closed. **No deny-lists**: the policy language is deliberately monotone grant-only, which makes policy review additive (a new grant can only widen). Deny-lists and attribute-based predicates are §6.

A concrete policy from the financial-institution scenario of §5, in Postern's surface syntax:

```
grant CRM       on users_data        { id, name, region, age }
grant CardOps   on cards_data        { card_id, card_type, limit, activated }
grant FraudRisk on transactions_data { txn_id, card_id, amount, merchant, timestamp }
grant FraudRisk on users_data        { id, region }
```

Anything outside these grants is implicitly denied; `users_data.ssn`, `users_data.email`, and `cards_data.card_number` are never released regardless of which principal queries. The rewriter projects each plan's output schema down to the grant union under the querying principal and refuses plans whose filter predicates touch columns outside that union (§4).

Rewriter

```
rewrite cat P prin q :=
  if cat q.touched = [] then     none                             -- unknown relation
  else if ¬ q.filterCols ⊆ allow then  none                        -- forbidden filter col
  else
    some (Project q (q.schema cat ∩ allow))
  where allow := P.allowed prin q.touched
```

Post-hoc projection is the simplest algorithm that admits a clean soundness proof. Predicate-pushdown variants can be verified against this rewriter as a reference; we leave that to future work.

Capability-bounded data flow

The rewriter of §3 (Layer 1) constrains what data reaches the agent. A separate layer (henceforth *Layer 2*) constrains what the agent's code may do with the values released. Odersky et al. (Odersky et al. 2026) propose Scala 3 capture-checking as a type-level mechanism for this purpose: capabilities are first-class program variables, and the compiler tracks each function's *capture set* — the capabilities it may use — so that agent-generated code cannot perform an effect for which it does not hold a capability. Rust has no capture-checking. We mechanise a weaker analog by composing three pure-Rust constructions, each closing one face of the gap.

Sealed capability tokens. `Cap<'sc, C>` carries no public constructor and contains a `Sealed` field whose constructor is private to the crate. Forging a `Cap` is therefore a privacy violation rejected at compile time (verified by a `compile_fail` doctest in the crate). Carrier values `Tagged<'sc, T, C>` likewise have a private `value` field; the inner T is unreachable by direct field access (also verified by `compile_fail`).

Invariant brand lifetime. Both `Cap` and `Tagged` carry a brand parameter $'sc$, made invariant by `PhantomData<fn(&'sc ()) -> &'sc ()>`. The sole entry point to the layer is a scope combinator

`run<T, C, R, F>(value, f) -> R` where `F: for<'sc> FnOnce(Cap<'sc, C>, Tagged<'sc, T, C>) ->`

The universal quantification of $'sc$ together with $R :' static$ forces the closure's return type to be free of $'sc$, ruling out any path by which a `Cap` or `Tagged` might escape the scope. The construction is the same one used by `ghost-cell` for branded references. A third `compile_fail` doctest demonstrates that attempting to return the `Cap` from the closure is rejected.

Opaque-receipt sinks. Carrier extraction is governed by a fixed set of sink functions, each of which consumes both `Cap<'sc, C>` and `Tagged<'sc, T, C>` and returns a *receipt* type that contains no information derived from T beyond its serialised length:

```
pub fn to_llm<'sc, T, C, S>(cap: Cap<'sc, C>,
                            data: Tagged<'sc, T, C>,
                            serialize: S) -> LlmAck
  where S: FnOnce(T) -> String;
```

The agent has no public method that returns raw T; the prior draft's `Tagged::release(cap) -> T` is removed in favour of the sink interface above. The only operations on *Tagged* that the agent's code can perform are `map` and `and_then`, both of which preserve the brand and the kind.

The agent-facing surface is `#![no_std]`. The crate is partitioned so that the gateway-side integration with `postern-core` lives behind a `gateway` feature flag; the agent-facing types and operations (`Cap`, `Tagged`, `run`, `sinks`) depend only on `core` and `alloc`. A downstream agent crate declaring `#![no_std]` and depending on `postern-guardrail` with `default-features = false` therefore has no link-level access to `std::println!`, `std::process::exit`, network sockets, or filesystem APIs, and the only side-effect channels available are those reached through the sanctioned sinks.

Residual. Inside a `map` closure body the agent has temporary access to a value of type T; Rust does not bound what that body may do with the value. Three classes of side-channel survive: `panic!` with formatted strings, timing observed by the host, and stash in thread-local storage when $T :' static$. We mitigate the obvious thread-move attack by making both `Cap` and `Tagged` `!Send + !Sync` via the `*const ()` phantom, but a determined adversary inside the agent runtime is out of scope for the present construction. A genuinely tight analog of capture-checking in Rust appears to require either a custom lint over closure bodies or a Wasm-class sandbox; both directions are discussed in §6.

Formal model

The development is mechanised in `verifier/lean/Postern.lean`; the per-theorem axiom set is reported by `CheckAxioms.lean`. We write *rewrite cat P p q* for the rewriter applied to catalog *cat*, policy P, principal p, and plan q. The output type is *Option Plan*; *none* denotes explicit refusal.

Theorem 1 (output-column soundness, `rewrite_sound`). For every cat, P, p, q, q', if $\mathit{rewrite}\ \mathit{cat}\ P\ p\ q = \mathit{some}\ q'$, then for every column $c \in \sigma(q')$, $c \in P.\mathit{allowed}\ p\ \mathit{touched}(q)$.

Theorem 2 (filter-predicate soundness, `rewrite_filter_sound`). Under the same hypothesis, every column read by a *Filter* predicate inside q' is also in $P.\mathit{allowed}\ p\ \mathit{touched}(q)$. Theorems 1 and 2 together rule out the side-channel in which a principal lacking read access to column c uses it as a row selector without projecting it.

Theorems 3 and 4 (`rewrite_schema_subset`, `rewrite_no_new_columns`). The output schema is contained in the input schema. Equivalently, $c \notin \sigma(q)$ implies $c \notin \sigma(q')$.

Theorem 5 (idempotence, `rewrite_idempotent`). If $\mathit{rewrite}\ \mathit{cat}\ P\ p\ q = \mathit{some}\ q'$ and $\mathit{rewrite}\ \mathit{cat}\ P\ p\ q' = \mathit{some}\ q''$, then $\sigma(q'') = \sigma(q')$ as sets. The rewriter is a closure operator on schemas.

Theorem 6 (monotonicity in the policy, `rewrite_monotone`). If $P.\mathit{allowed}\ p\ r \subseteq P'.\mathit{allowed}\ p\ r$ for every p and r, then the output schema under P is contained in the output schema under P'. Strengthening the policy can only widen the released set.

Theorem 7 (touched-relation preservation, `rewrite_touched`). $\mathit{touched}(q') = \mathit{touched}(q)$.

Theorem 8 (refusal under unknown relation, `rewrite_refuses_unknown`). $\mathit{cat}\ \mathit{touched}(q) = []$ implies $\mathit{rewrite}\ \mathit{cat}\ P\ p\ q = \mathit{none}$. A relation absent from the catalog is rejected explicitly rather than reduced to an empty output schema — relevant under catalog drift, where the physical store may diverge from the catalog.

Theorem 9 (refusal under forbidden filter, `rewrite_refuses_forbidden_filter`). If $c \in \mathit{filterCols}(q)$ and $c \notin P.\mathit{allowed}\ p\ \mathit{touched}(q)$, then $\mathit{rewrite}\ \mathit{cat}\ P\ p\ q = \mathit{none}$. The contrapositive of Theorem 2.

`CheckAxioms.lean` audits the axiom dependencies of each theorem. The set is bounded by {`propext`, `Quot.sound`}, Lean~4's foundational axioms; the proofs of `rewrite_touched` and `rewrite_refuses_unknown` depend on no axioms. No proof uses `sorry`, and no user-supplied `axiom` declarations are introduced.

Implementation and conformance testing

The Rust implementation in `prototype/crates/postern-core` mirrors the Lean types and the *rewrite* function structurally. The conformance harness `postern-diff` consumes a JSON corpus emitted by `lake exe postern-corpus` and asserts three equalities per case: that the Rust outcome kind (*accept* / *refuse*) matches the Lean reference; that, on accept, the rewritten plan, output schema, predicate read-set, and touched relation are structurally equal to the Lean reference; and that the input plan's *filterCols* matches the Lean auxiliary, independent of the rewriter.

JSON-corpus conformance is preferred to Lean-to-Rust extraction on the grounds that the corpus interface is stable across compiler-version churn in both languages and that divergence manifests as a CI failure rather than a build failure.

The corpus comprises 18 cases: seven behavioural cases drawn from the financial-institution scenario of §5, four refusal regressions for known attack shapes (filter-on-forbidden-column, unknown-relation, two nested forbidden-filter variants), and seven policy-language edge cases (empty policy, duplicate grants, catalog-absent columns, case-sensitive principal, trailing-whitespace principal, nonexistent project column, nested *Project* narrowing). All eighteen pass on the current Rust implementation.

Evaluation: a financial institution with three principals

We evaluate the artifact on the Kaggle `transactions-fraud-datasets` schema. The policy is reproduced in `scenarios/financial-institution/policy.postern`; the principal cases are summarised below.

principal	plan	outcome	rewritten schema
`CRM`	`Scan users_data`	accept	`id, name, region, age`
`CRM`	`Project [ssn,email]` over above	accept	$\emptyset$ (over-projection collapses)
`CRM`	`Filter on ssn`	**refuse**	—
`CardOps`	`Scan users_data` (cross-dept)	accept	$\emptyset$ (no matching grant)
`FraudRisk`	`Scan users_data`	accept	`id, region` (minimum-necessary)
`Marketing`	`Scan users_data` (unknown prin.)	accept	$\emptyset$ (empty allow)
`CRM`	`Scan credit_bureau_imports`	**refuse**	— (unknown relation)

Each row corresponds to a corpus case in the conformance harness; the rows annotated *refuse* exercise Theorems 8 and 9 of §4.

Related work

We are not aware of prior work that establishes a mechanised soundness theorem for a plan-level rewriter in an LLM-agent- facing lakehouse setting. The closest landmarks fall into four groups.

Verified authorization decision procedures. Cedar (Cutler et al. 2024) formalises and proves the soundness of an authorization-decision function in Lean. The axis of verification differs from ours: Cedar establishes authorize(*request*) $\in \{allow, deny\}$ correctly classifies a per-call request, whereas we establish that the *output of a plan transformation* is contained in the policy-allowed set. The two are complementary; we adopt the Cedar style of Lean-mechanised denotational semantics for our policy.

Capability-based enforcement runtimes. SEAL (Sadeghi et al. 2023) provides capability-based access control for analytic workloads at the runtime level; the policy core is not mechanically verified. Our development is the dual: the policy core is verified, and the runtime is correspondingly lighter. Biscuit (Clever Cloud and contributors 2024) provides the deployed capability-token distribution mechanism we assume on the front end.

Information-flow control for database-backed applications. Jeeves (Yang et al. 2012), Jacqueline (Yang et al. 2016), and the faceted-execution line (Schoepe et al. 2016) enforce IFC inside ORM-backed applications using a faceted-value runtime discipline. They predate the LLM-agent threat model and do not target the lakehouse setting; we view them as the closest PL-side relatives of Layer 2 of our development.

Defences for LLM-agent prompt injection. AgentDojo (Debenedetti et al. 2024) and CaMeL (Debenedetti et al. 2025) develop capability-flow defences at the agent boundary. Our development is complementary: the rewriter of §3–§4 enforces a policy at the lake-facing boundary on plans; their constructions enforce analogous properties on the agent's own emitted code. Closest to our Layer 2 specifically, Odersky et al. (Odersky et al. 2026) propose Scala~3 capture-checking as the type-level mechanism for tracking capabilities through agent code; we adapt the same intuition under Rust's weaker type-system commitments (§3).

Deployed alternatives we improve upon. PostgreSQL row security policies (PostgreSQL Global Development Group 2024) and equivalent CLS facilities require per-engine integration and do not compose across the heterogeneous ingest paths typical of lakehouse deployments. Open Policy Agent (Cloud Native Computing Foundation 2021) is general-purpose but does not reason about query outputs at the plan level. Tenant segregation forfeits cross-source analytics and is the silo case we discuss in §1.

Open challenges and future work

Three extensions of the Lean development are the natural next research questions.

Cross-relation joins. The Plan IR is single-relation. The Rust implementation handles joins by per-leg rewriting but the composition is not under proof. The conjecture is a theorem of the form

$$\mathit{rewrite\_sound\_join} : \mathit{accept}(q_1) \wedge \mathit{accept}(q_2) \implies \sigma(\mathit{rewrite}(\mathit{Join}(q_1, q_2))) \subseteq \bigcup_i P.\mathit{allowed}\, p\ \mathit{touched}(q_i)$$

on a Plan IR extended with a *Join* constructor. The join-key leak — joining on a column c without projecting it, which leaks c's value distribution — mirrors the filter side-channel and admits the analogous coverage condition.

Aggregation with a differential-privacy boundary. A principal may be permitted to read SUM(*amount*) without permission to read individual rows. The rewriter extension here is straightforward in shape but admits a non-trivial soundness statement once the differential-privacy boundary is parameterised; SEAL (Sadeghi et al. 2023) and the faceted line (Schoepe et al. 2016) are the closest reference points.

Capability attenuation inside the proof. The Lean development takes *Principal* as a flat string and assumes the gateway has already verified the bearer of a biscuit token. Modelling biscuit's Datalog-based attenuation, expiry, and audience checks inside the Lean proof lifts the principal- extraction row out of the trusted base — a substantial strengthening of the artifact's overall claim. Adjacent open problems include catalog-integrity attestation and plan- integrity in transit.

Reproducibility

```
verifier/lean/   Lean 4 spec + theorems + corpus emitter
prototype/       Rust workspace: postern-core, postern-diff
scenarios/       Financial-institution case study
paper/           This document
scripts/         reproduce.sh - chains everything
```

Toolchains: **Lean 4.29.1** (pinned in `verifier/lean/lean-toolchain`), **Rust stable** (tested 1.93). Single command:

```
scripts/reproduce.sh
```

Expected output ends with `18/18 cases pass (Lean reference == Rust impl)` and an axiom audit showing only `propext` and `Quot.sound`. Runs in under two minutes on an M-series Mac on a warm cache.

References

Anthropic. 2024. *Model Context Protocol.* Https://modelcontextprotocol.io.

Clever Cloud and contributors. 2024. *Biscuit: A Bearer Token with Datalog-Based Offline Authorization.* Https://www.biscuitsec.org.

Cloud Native Computing Foundation. 2021. "Open Policy Agent and the Rego Language." *CNCF Graduated Project Documentation.*

Cutler, Joseph W., Craig Disselkoen, Aaron Eline, et al. 2024. "Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization." *Proc. ACM Program. Lang.* 8 (OOPSLA1).

Debenedetti, Edoardo et al. 2025. *Defeating Prompt Injections by Design.*

Debenedetti, Edoardo, Jie Zárate, and Tramer Florian Tramer. 2024. "Agent-Dojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." *Advances in Neural Information Processing Systems (NeurIPS).*

Moura, Leonardo de, and Sebastian Ullrich. 2021. *The Lean 4 Theorem Prover and Programming Language.* Https://lean-lang.org.

Odersky, Martin, Yichen Zhao, Yifan Xu, Oliver Bračevac, and Cao Nguyen Pham. 2026. *Tracking Capabilities for Safer Agents.* https://arxiv.org/abs/2603.00991.

PostgreSQL Global Development Group. 2024. *Row Security Policies.* Https://www.postgresql.org/docs/current/ddl-rowsecurity.html.

Raasveldt, Mark, and Hannes Mühleisen. 2019. *DuckDB: An in-Process SQL OLAP Database Management System.* Https://duckdb.org.

Sadeghi, Ahmad-Reza et al. 2023. "SEAL: Capability-Based Access Control for Data-Analytic Scenarios." *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies (SACMAT).* https://doi.org/10.1145/3589608.3593838.

Schoepe, Daniel, Musard Balliu, Benjamin C. Pierce, and Andrei Sabelfeld. 2016. "Explicit Secrecy: A Policy for Taint Tracking." *IEEE European Symposium on Security and Privacy (EuroS&P).* https://doi.org/10.1109/EuroSP.2016.14.

Yang, Jean, Travis Hance, Thomas H. Austin, Armando Solar-Lezama, Cormac Flanagan, and Stephen Chong. 2016. "Precise, Dynamic Information Flow for Database-Backed Applications." *PLDI*. https://doi.org/10.1145/290808 0.2908098.

Yang, Jean, Kuat Yessenov, and Armando Solar-Lezama. 2012. "A Language for Automatically Enforcing Privacy Policies." *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. https://doi.org/10.1145/2103656.2103669.